

Animație text

-FPGA Nexys2-

Proiect realizat de Dragoș Marcean și Mile László

Facultatea de automatică și calculatoare

Anul I de studiu

Grupa 30212

Profesor Indrumător

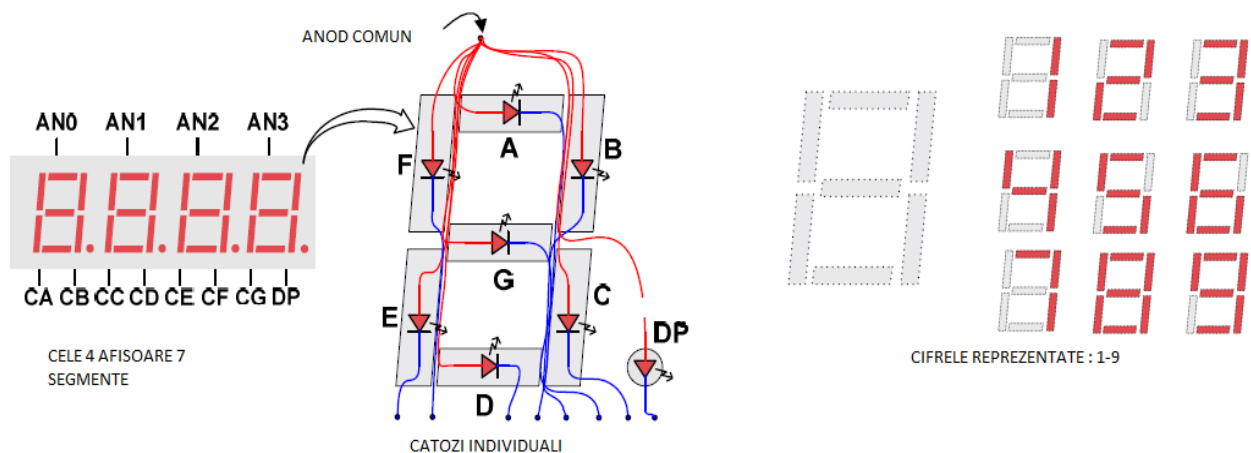
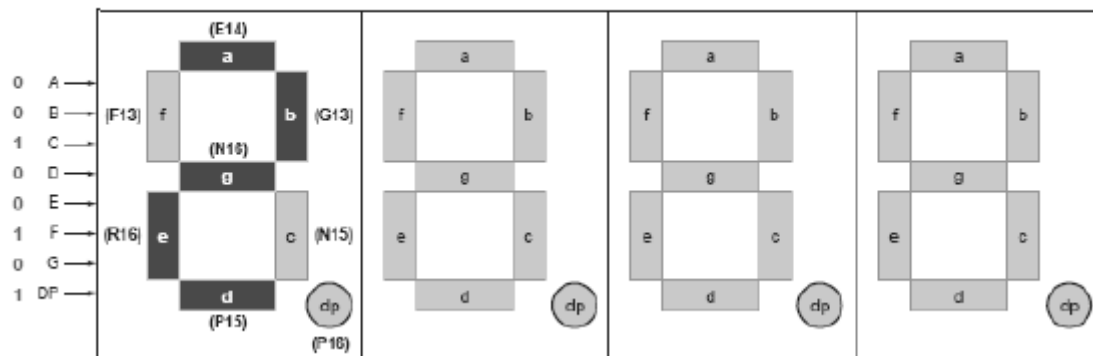
Sl. Dr. Ing Lucia Văcariu

PLACA NEXYS-2



Fiind produsă de compania Digilent Inc., placa **NEXYS-2** are în principiu componente principale similare cu cele ale plăcii Spartan3 Starter Kit: **cele 4 afișoare 7 segmente, cele 8 comutatoare cu 2 stări, cele 4 comutătoare de tip push-button și LED-urile.**

a) cele 4 afișoare 7 segmente



-după cum se observă din imagine, afișoarele au cele 7 leduri reprezentând segmentele, si un simbol **dp** reprezentând punctul decimal

-fiecare are un anod activ pe 0 (funcționează numai atunci când sunt setate pe 0 logic)

Conexiunile dintre dispozitivul FPGA si afișajul cu 7 segmente (active pe 0)
Segmentul Pinul dispozitivului FPGA

A	L18
B	F18
C	D17
D	D16
E	G14
F	J17
G	H14
DP	C17

Semnalele de control al anodului (active pe 0)

Anode Control	AN3	AN2	AN1	AN0
FPGA Pin	F15	C18	H17	F17

b) Cele opt comutatoare cu două poziții

Conexiunile comutatoarelor la pinii dispozitivului FPGA

Comutator	SW7	SW6	SW5	SW4	SW3
SW2	SW1	SW0			
Pin FPGA	R17	N17	L13	L14	K17
K18	H18	G18			

c) Cele patru comutatoare de tip „push button”

Conexiunile comutatoarelor de tip „push button” la pinii dispozitivului FPGA

Push Button	BTN3	BTN2	BTN1	BTN0
Pin FPGA	B18	D18	E18	H13

d) LED-urile

Conexiunile LED-urilor la pinii dispozitivului FPGA Spartan3

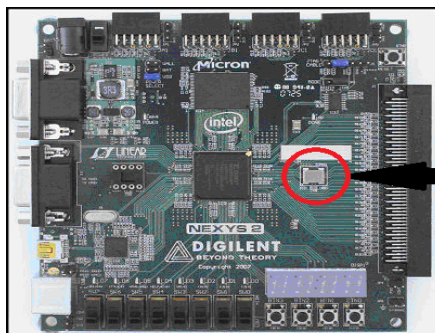
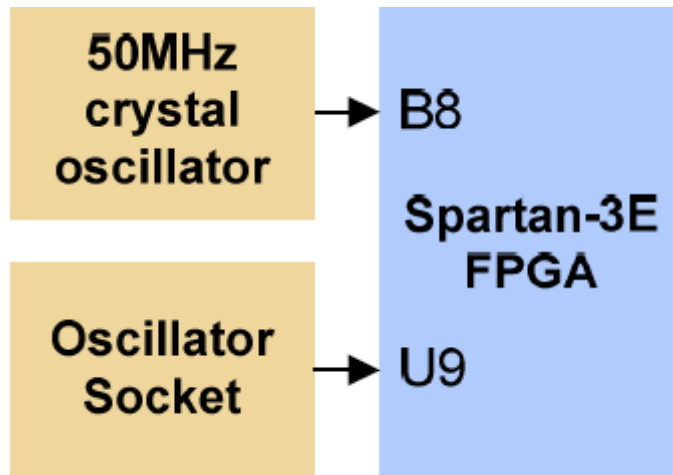
LED	LD7	LD6	LD5	LD4	LD3
LD2	LD1	LD0			
Pin FPGA	P4	E4	P16	E16	K14
K15	J15	J14			

Programarea plăcii

Pentru programarea plăcii nu se mai poate folosi utilitarul **IMPACT** din mediul ISE Foundation, ci se va folosi programul **ADEPT** (mai precis modulul **ExPort** al acestuia).

Clock-ul integrat în placa FPGA

Placa NEXYS-2 include un oscilator de quartz interior de **50MHz** și o cutie pentru un al doilea oscilator. Semnalele de tact de la oscilatorii se conectează la pinii de tact global. Astfel se poate realiza divizarea de frecvență precum și multiplicarea acestuia pentru semnale de tact de frecvență variabilă.



-clock-ul integrat in placă

Componentele logice folosite pentru realizarea proiectului

Multiplexorul 4:1

Cu acest multiplexor vom putea alege animația dorită din cele patru. Utilizatorul va putea alege acesta practic prin folosirea a două comutatoare de două poziții din cele 8 disponibile (mai precis primul și al doilea):



Animația1: comutatoarele fiind: 00 - clipire literă cu literă: C _ _ _ _ O _ _ _ _ L _ _ _ _ A

Animația2: comutatoarele fiind: 01 - deplasare stânga-dreapta

Animația3: comutatoarele fiind: 10 - deplasare dreapta-stânga

Animația4: comutatoarele fiind: 11 - clipire toate

Memoriile ROM

Textul ce va trebui să fie afișat este hard-codat in codul **vhdl**. Folosim memoria ROM pentru a reprezenta literele **alfabetului englez** de la **-a-** la **-z-**.

Reprezentarea se face ținând cont de faptul că **ledurile** segmentelor sunt **active pe 0 logic**. Utilizatorul va putea alege textul dorit făcând modificări in codul **vhdl** care va fi prezentat la sfârșitul documentației. Modificările necesare sunt:

- se schimbă valoarea vectorului **cuv()** alegând dintre familia de vectori **I()** care reprezinta litera respectivă.

Numărătorul pe 26 de biți

Acesta este cel mai important element al animației fără de care utilizatorul n-ar avea posibilitatea de a vedea animația respectivă. Clock-ul plăcii **NEXYS-2** funcționează pe **50MHz**, de aceea avem nevoie de o modalitate de a incetini timpul de execuție. Folosind un numărător care va număra o secundă (sau dacă e cazul mai mult sau mai puțin) putem vedea cu ochiul liber schimbările efectuate pe afișoarele folosite.

Registrul de deplasare

Registrul de deplasare este folosit pentru a realiza deplasarea literelor textului de afișat într-o direcție ales de către utilizator.

Codul_vhdl

```

library work, std;
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

use std.standard.all;

entity segment is
    port
        (an: inout std_logic_vector (3 downto 0):="1110";
         segmente: out std_logic_vector (6 downto 0);
         clk: in bit;
         anim: in std_logic_vector(1 downto 0)
        );
end segment;

architecture comportament of segment is
    signal counter: std_logic_vector(25 downto 0):="000000000000000000000000";
    signal counter2: std_logic_vector(15 downto 0):="0000000000000000";
    type memo is array (24 downto 0) of std_logic_vector (6 downto 0);
    type mem is array (15 downto 0) of std_logic_vector (6 downto 0);
    constant l: memo:=
0 => "1111111", -- spatiu      primul este dp care il punem pe 1
1 => "0001000", -- A          --LA ASSIGN PACKAGE PINS SA PUNEM INVERS PIN URILE
2 => "1100000", -- B
3 => "1000110", -- C
4 => "1000010", -- D
5 => "0000110", -- E
6 => "0111000", -- f
7 => "0100000", -- g
8 => "1101000", -- h
9 => "1001111", -- i
10 => "1000011", -- j
11 => "1111000", -- k
12 => "1000111", -- l
13 => "0101010", -- m
14 => "1101010", -- n
15 => "1000000", -- o
16 => "0011000", -- p
17 => "0010000", -- r
18 => "0100100", -- s
19 => "1111010", -- t
20 => "1000001", -- u
21 => "1100011", -- v
22 => "1000000", -- w
23 => "1001000", -- x
24 => "0010010"  -- z
    );

```



```

constant cuv: mem:=
0 =>1(3),      --C
1 =>1(15),     --O
2 =>1(12),     --L
3 =>1(1),      --A
-- COLA
4 =>1(15),     --O
5 =>1(12),     --L
6 =>1(1),      --A
7 =>1(3),      --C
-- OLAC
8 =>1(12),     --L
9 =>1(1),      --A
10 =>1(3),     --C
11 =>1(15),    --O
-- LACO
12 =>1(1),     --A
13 =>1(3),     --C
14 =>1(15),    --O
15 =>1(12));--L
-- ACOL
--Dupa care se reia de la 0

begin --arhitectura

animatie: process(clk)  -- clk legam la B8
variable i: integer :=0;
variable j: integer :=15;
begin
    if clk='1' and clk'event then --
        case anim is        -- dreapta - stanga
            ----- prima animatie
                when "10" => -----
                    if (counter2="1111000000000000") then
                        if(AN(0)='0') then
                            AN(0)<='1';
                            segmente<=cuv(i+3);
                            AN(3)<='0';
                        elsif (AN(3)='0') then
                            AN(3)<='1';
                            segmente<=cuv(i+2);
                            AN(2)<='0';
                        elsif (an(2)='0') then
                            an(2)<='1';
                            segmente<=cuv(i+1);
                            an(1)<='0';
                        elsif (an(1)='0') then
                            an(1)<='1';
                            segmente<=cuv(i);
                            an(0)<='0';
                        end if; -- AN
                    end if;
                end if;
            end case;
        end if;
    end process;
end begin;

```



```

end if;          -- count 2
if (counter2="1111000000000000") then counter2 <= (others =>'0'); end if;
    counter2 <= counter2 + 1;
    counter <= counter + 1;
if(i=16) then i:=0; end if;
if (counter="101111010111100001011111") then i:=i+4; counter <= (others =>'0'); end if;
----- prima animatie

----- a doua animatie
when "01" => -- stanga - dreapta
if (counter2="1111000000000000") then
if(AN(0)='0') then
    AN(0)<='1';
    segmente<=cuv(j);
    AN(3)<='0';
elseif (AN(3)='0') then
    AN(3)<='1';
    segmente<=cuv(j+1);
    AN(2)<='0';
elseif (an(2)='0') then
    an(2)<='1';
    segmente<=cuv(j+2);
    an(1)<='0';
elseif (an(1)='0') then
    an(1)<='1';
    segmente<=cuv(j+3);
    an(0)<='0';
end if; -- AN
end if; -- count 2
if (counter2="1111000000000000") then counter2<="0000000000000000"; end if;
    counter2 <= counter2 + 1;
    counter <= counter + 1;
if(j<0) then i:=15; end if;
if (counter="101111010111100001011111") then j:=j-4; counter <= (others =>'0'); end if;
----- a doua animatie

----- a treia animatie
when "11" => -- clipire toate
if (counter2="1111000000000000") then
if(AN(0)='0') then
    AN(0)<='1';
    segmente<=cuv3(i);
    AN(3)<='0';
elseif (AN(3)='0') then
    AN(3)<='1';
    segmente<=cuv3(i+1);
    AN(2)<='0';
elseif (an(2)='0') then
    an(2)<='1';
    segmente<=cuv3(i+2);
    an(1)<='0';

```

```

        elsif (an(1)='0') then
            an(1)<='1';
            segmente<=cuv3(3);
            an(0)<='0';
        end if; -- AN
    end if; -- count 2
    if (counter2="1111000000000000") then counter2 <= (others =>'0'); end if;
    counter2 <= counter2 + 1;
    counter <= counter + 1;
    if(i=8) then i:=0; end if;
    if (counter="1011111010111100001011111") then i:=i+4; counter <= (others =>'0'); end if;
    ----- a treia animatie

    ----- a patra animatie

    when "00" =>
        if (counter2="1111000000000000") then
            if(AN(0)='0') then
                AN(0)<='1';
                segmente<=cuv2(i+3);
                AN(3)<='0';
            elsif (AN(3)='0') then
                AN(3)<='1';
                segmente<=cuv2(i+2);
                AN(2)<='0';
            elsif (an(2)='0') then
                an(2)<='1';
                segmente<=cuv2(i+1);
                an(1)<='0';
            elsif (an(1)='0') then
                an(1)<='1';
                segmente<=cuv2(i);
                an(0)<='0';
            end if; -- AN
        end if; -- count 2
        if (counter2="1111000000000000") then counter2 <= (others =>'0'); end if;
        counter2 <= counter2 + 1;
        counter <= counter + 1;
        if(i=16) then i:=0; end if;
        if (counter="1011111010111100001011111") then i:=i+4; counter <= (others =>'0'); end if;
        ----- a patra animatie
        when others =>
            an <="0000";
            segmente <= l(15);

        end case;
    end if; --clk
end process animatie;
end architecture;

```

Alfabetul folosit:

```
"1111111", -- spatiu  
"0001000", -- A  
"1100000", -- B  
"1000110", -- C  
"1000010", -- D  
"0000110", -- E  
"0111000", -- f  
"0100000", -- g  
"1101000", -- h  
"1001111", -- i  
"1000011", -- j  
"1111000", -- k  
"1000111", -- l  
"0101010", -- m  
"1101010", -- n  
"1000000", -- o  
"0011000", -- p  
"0010000", -- r  
"0100100", -- s  
"1111010", -- t  
"1000001", -- u  
"1100011", -- v  
"1000000", -- w  
"1001000", -- x  
"0010010", -- z
```

Semnificația notațiilor

- *an ->anodurile (vector de 4 componente care reprezintă cele 4 anoduri)*
- *segmente->ieșirile pentru afișoarele 7 segmente (vector de 7 componente)*
- *clk->clock-ul de la placuță*
- *counter->numărătorul care va întârzia execuția cu o secundă*
- *anim->animațiile (intrări care permit utilizatorului să selecteze animația dorită)*

Proiectul nr.9 animația text

Să se proiecteze o reclamă publicitară cu animații multiple. Se vor folosi cele 4 afișaje cu 7 segmente de pe plăcuțele Digilent Nexys-2 și Starter. Textul de afișat va fi hard-codat (în momentul sintezei proiectului se știe textul reclamei, care va fi încorporat în arhitectura hardwer). Reclama va avea mai multe regimuri de funcționare (minimum 4) ce vor putea fi selectate de către utilizator, de la comutatoarele plăcuței FPGA. Se vor folosi oscilatorul de cuarț încorporat în plăcuța FPGA (semnalul de clock va trebui să fie desigur divizat). În ceea ce privește regimurile de funcționare, se vor folosi ca sursă de inspirație PPT sau reclamele existente (vezi farmacii). Exemple de regimuri de funcționare: - curgerea- scrisului de la dreapta spre stânga, pâlpâire, afișaj litera cu litera sau simbol cu simbol).

Deoarece pe un afișaj cu 7 segmente nu se pot reprezenta toate literele, se va crea un alfabet maximal și mesajele vor fi compuse din simbolurile acelui. Proiectul va fi realizat de 2 studenți.

Cuprins

PAG 1.....	Titlu și nume
PAG2.....	Cerința Proiectului
PAG3-5.....	Placa NEXYS-2
PAG 6.....	Schema bloc
PAG 7.....	Componentele folosite
PAG 8.....	Alfabetul(cod) și semnificația notațiilor
PAG 9-12.....	Cod-ul VHDL
PAG 13-14.....	Animațiile
PAG 15-19.....	Instrucțiuni de utilizare si explicații
PAG 10.....	Alfabetul(litere)
PAG 21	Posibilități de dezvoltare ulterioară

Animațiile

Animația 1

Pentru ca placa FPGA să efectueze animația numită -deplasarea dreapta-stânga-, utilizatorul trebuie să modifice valorile de 00 logic de pe switch-uri pe valorile de 10 logic. Clock-ul integrat, deoarece funcționează pe 50MHz, trebuie să îl încetinim cu ajutorul semnalelor de counter, respectiv de counter2. După ce o secundă a trecut, încărcăm pe placă un alt cuvânt, astfel încât ca deplasarea să poată fi efectuată.



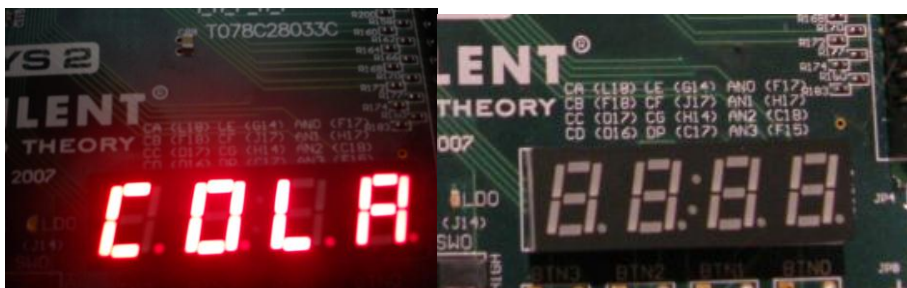
Animația 2

Animația a doua este asemenea primei cu diferența că deplasarea se efectuează de la stânga spre dreapta. Ideea de bază este asemeni animației prezentate anterior. Codul pentru această animație este 01 logic.



Animația 3

A treia animație poartă numele de -clipire toate-. Textul dorit este încărcat pe afișoarele iar după ce am numărat până la 50 de milioane (echivalent cu 1 secundă) am încărcat pe ele spațiu, adică codul pentru ca afișoarele să pară a fi stinse. Pentru a obține această animație schimbăm valorile pe switch-uri pe 11 logic.



Animația 4

Această animație constă în apariția pe rând, câte una a tuturor literelor, fiecare pe altă poziție. Aceasta se face încărcând o literă și trei spații.

```
CONSTANT cuv2: mem:=0
0 =>1(3),      --C
1 =>1(0),      --
2 =>1(0),      --
3 =>1(0),      --
-- COLA
4 =>1(0),      --
5 =>1(15),     --O
6 =>1(0),      --
7 =>1(0),      --
-- OLAC
8 =>1(0),      --
9 =>1(0),      --
10 =>1(12),    --L
11 =>1(0),     --
-- LACO
12 =>1(0),     --
13 =>1(0),     --
14 =>1(0),     --
15 =>1(1));    --A
```

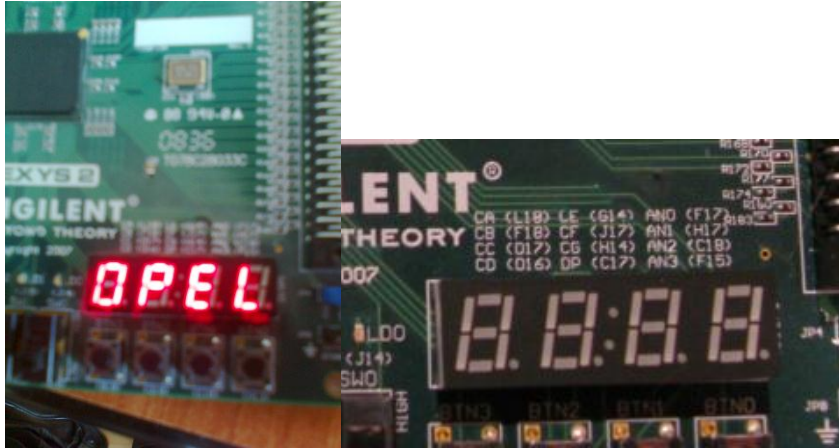
```
when "00" =>
if (counter2="1111000000000000") then
if(AN(0)='0') then
    AN(0)<='1';
    segmente<=cuv2(i+3);
    AN(3)<='0';
elseif (AN(3)='0') then
    AN(3)<='1';
    segmente<=cuv2(i+2);
    AN(2)<='0';
elseif (an(2)='0') then
    an(2)<='1';
    segmente<=cuv2(i+1);
    an(1)<='0';
elseif (an(1)='0') then
    an(1)<='1';
    segmente<=cuv2(i);
    an(0)<='0';
    end if; -- AN
end if; -- count 2
if (counter2="1111000000000000") then counter2 <= (others =>'0'); end if;
    counter2 <= counter2 + 1;
    counter <= counter + 1;
if(i=16) then i:=0; end if;
if (counter="1011111010111100001011111") then i:=i+4; counter <= (others =>'0'); end if;
```


Posibilități de dezvoltare ulterioară

- **Folosirea a mai puține memorii pentru animații**
- **Introducerea de cuvinte de mai multe litere**
- **Animații mai dinamice care să atragă atenția asupra reclamei**

Instrucțiuni de utilizare si explicații

1. Animația 1 --- clipire tot cuvântul



```

when "11" => -- clipire toate
  if (counter2="1111000000000000") then
    if(AN(0)='0') then
      AN(0)<='1';
      segmente<=cuv3(i);
    elsif (AN(3)='0') then
      AN(3)<='1';
      segmente<=cuv3(i+1);
    elsif (AN(2)='0') then
      AN(2)<='1';
      segmente<=cuv3(i+2);
    elsif (AN(1)='0') then
      AN(1)<='1';
      segmente<=cuv3(3);
      AN(0)<='0';
    end if; -- AN
  end if; -- count 2
  if (counter2="1111000000000000") then counter2 <= (others =>'0'); end if;
  counter2 <= counter2 + 1;
  counter <= counter + 1;
  if(i=8) then i:=0; end if;
  if (counter="101111101011100001011111") then i:=i+4; counter <= (others =>'0'); end if;

```

```

constant cuv3: mem2:= (
  0 => 1(15),      --O
  1 => 1(16),      --P
  2 => 1(5),       --E
  3 => 1(12),      --L
  -- COLA
  4 => 1(0),       --
  5 => 1(0),       --
  6 => 1(0),       --
  7 => 1(0),       --
  ----

```

Pentru această animație am creat memoria de dimensiunea **ROM 8x7** care conține cele **4 litere** ale cuvântului și **4 spații**. De menționat că pentru fiecare animație, cuvântul se poate modifica oricând, schimbând adresele primite de la memoria "1". Pentru exemplul nostru am luat cuvântul **OPEL**, care apare și dispare la interval de aproximativ **1 secundă**.

Counter-ul are rolul de a întârzia modificarea variabilei *i* care are rolul de a încărca fie cuvântul, fie spațiul. **Clock-ul** este de **50MHZ**, iar numărând până la aproximativ **50.000.000**, se întârzie execuția cu **1 secundă**,

În cadrul primei instrucțiuni **if** se va afișa pe afișor cuvântul nostru. Se deschide pe rând câte un anod, restul fiind închise, deoarece dacă am avea mai multe anoduri deschise, s-ar afișa același caracter în mai multe locuri. Schimbarea de la o instrucțiune de test la alta se face foarte repede și ochiul uman nu percepe închiderea și deschiderea anodurilor.

Pentru a selecta această animație, utilizatorul va da valorile de **1 logic** la primele 2 întrerupătoare de pe plăcuța **Nexys2**, cu codul **R17** și **N17**.

2. Animația 2 – deplasare dreapta - stânga



```

when "10" => ----- MERGE PERFEEECT!!!!
if (counter2="1111000000000000") then
  if (AN(0)='0') then
    AN(0)<='1';
    segmente<=cuv(i);
    AN(3)<='0';
  elsif (AN(3)='0') then
    AN(3)<='1';
    segmente<=cuv(i+1);
    AN(2)<='0';
  elsif (AN(2)='0') then
    AN(2)<='1';
    segmente<=cuv(i+2);
    AN(1)<='0';
  elsif (AN(1)='0') then
    AN(1)<='1';
    segmente<=cuv(i+3);
    AN(0)<='0';
  end if;
  -- AN
end if;
if (counter2="1111000000000000") then counter2 <= (others => '0'); end if;
  counter2 <= counter2 + 1;
  counter <= counter + 1;
if (i=16) then i:=0; end if;
if (counter="10111110101111000010111111") then i:=i+4; counter <= (others => '0'); end if;
----- prima animatie
  
```

```

constant cuv: mem:=(
0 =>1(3),      --C
1 =>1(15),     --O
2 =>1(12),     --L
3 =>1(1),      --A
-- COLA
4 =>1(15),     --O
5 =>1(12),     --L
6 =>1(1),      --A
7 =>1(3),      --C
-- OLAC
8 =>1(12),     --L
9 =>1(1),      --A
10=>1(3),      --C
11=>1(15),     --O
-- LACO
12=>1(1),      --A
13=>1(3),      --C
14=>1(15),     --O
15=>1(12)); --L
  
```

Vom utiliza o memorie **ROM 16x7**, în care vor fi încărcate toate cele 4 faze ale deplasării. Pentru aceasta am ales cuvântul **COLA**.

Counter-ul va întârzia încărcarea următoarei faze a deplasării cu o secundă, la fel numărând până la 50.000.000. Se adună la variabila "i" 4 unități și pe afișor vor apărea următoarele 4 litere și vom avea iluzia deplasării. La fel ca la animația precedentă, ochiul nu va percepe închiderea și deschiderea anodurilor și vor apărea toate deschise în același timp.

Utilizatorul va putea accesa această animație dând valorile **1 și 0** pe întrerupătoarele **R17 și N17**.

3. Animație 3 – deplasare stânga – dreapta



```

when "01" => -- stanga - dreapta
if (counter2="1111000000000000") then
if (AN(0)='0') then
  AN(0)<='1';
  segmente<=cuv(j);
  AN(3)<='0';
elseif (AN(3)='0') then
  AN(3)<='1';
  segmente<=cuv(j+1);
  AN(2)<='0';
elseif (AN(2)='0') then
  AN(2)<='1';
  segmente<=cuv(j+2);
  AN(1)<='0';
elseif (AN(1)='0') then
  AN(1)<='1';
  segmente<=cuv(j+3);
  AN(0)<='0';
end if;
-- AN
end if;
-- count 2
if (counter2="1111000000000000") then counter2<="0000000000000000"; end if;
  counter2 <= counter2 + 1;
  counter <= counter + 1;
if (j<0) then i:=15; end if;
if (counter="1011111010111100001011111") then j:=j-4; counter <= (others =>'0'); end if;
  
```

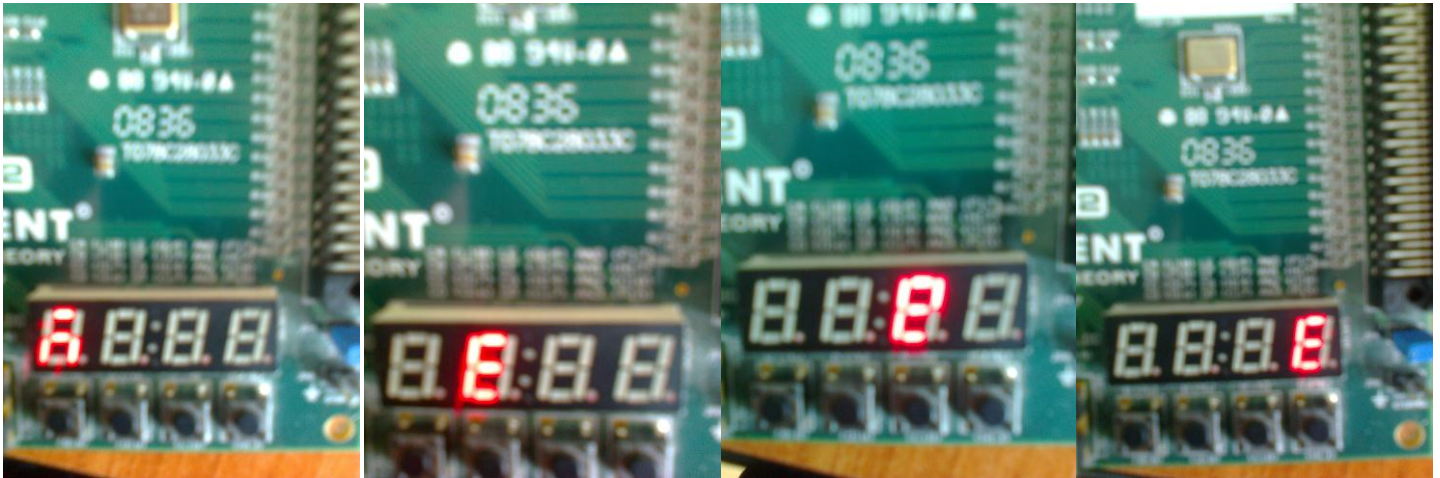
```

constant cuv: mem:=(
0 =>1(3),      --C
1 =>1(15),     --O
2 =>1(12),     --L
3 =>1(1),      --A
-- COLA
4 =>1(15),     --O
5 =>1(12),     --L
6 =>1(1),      --A
7 =>1(3),      --C
-- OLAC
8 =>1(12),     --L
9 =>1(1),      --A
10=>1(3),      --C
11=>1(15),     --O
-- LACO
12=>1(1),      --A
13=>1(3),      --C
14=>1(15),     --O
15=>1(12)); --L
  
```

Această animație este asemanatoare cu precedenta. Diferă doar **intrările date la intrerupătoare** și **ordinea în care se încarcă adresele** din memoria ROM 16x7.

Intrările pentru această animație sunt **0 și 1** pe **R17 și N17**.

4. Animatie 4 – clipire pe rand a literelor



```

when "00" =>
  if (counter2="1111000000000000") then
    if (AN(0)='0') then
      AN(0)<='1';
      segmente<=cuv2(i);
      AN(3)<='0';
    elsif (AN(3)='0') then
      AN(3)<='1';
      segmente<=cuv2(i+1);
      AN(2)<='0';
    elsif (AN(2)='0') then
      AN(2)<='1';
      segmente<=cuv2(i+2);
      AN(1)<='0';
    elsif (AN(1)='0') then
      AN(1)<='1';
      segmente<=cuv2(i+3);
      AN(0)<='0';
    end if; -- AN
  end if; -- count 2
  if (counter2="1111000000000000") then counter2 <= (others =>'0'); end if;
  counter2 <= counter2 + 1;
  counter <= counter + 1;
  if(i=16) then i:=0; end if;
  if (counter="101111010111100001011111") then i:=i+4; counter <= (others =>'0'); end if;
  
```

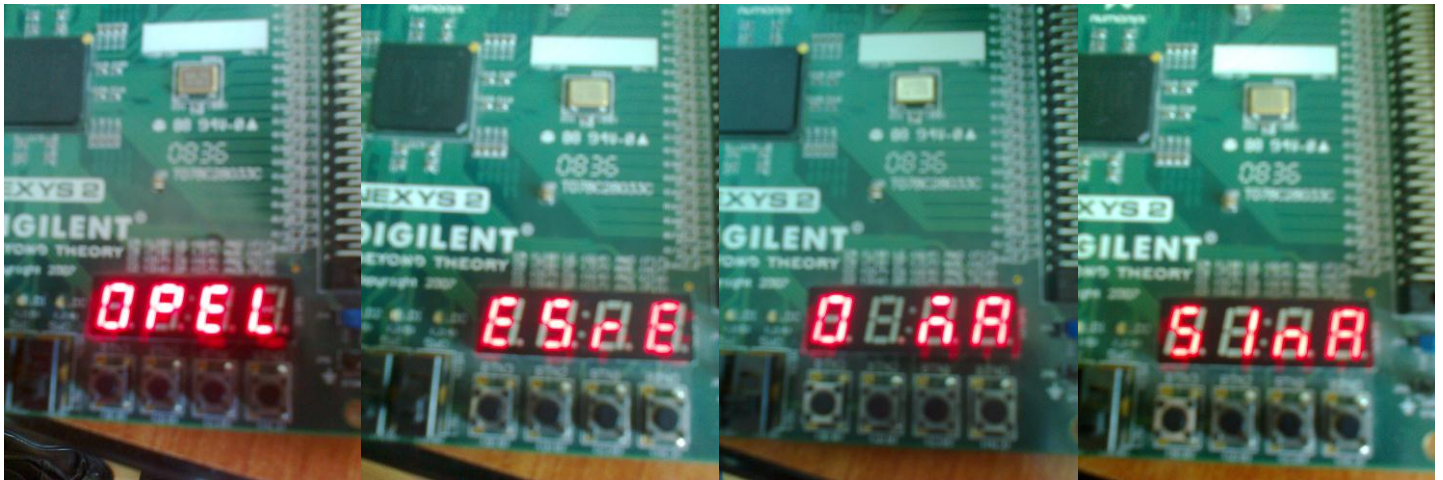
```

constant cuv2: mem:=(
  0 =>1(13),      --M
  1 =>1(0),       --
  2 =>1(0),       --
  3 =>1(0),       --
  -- COLA
  4 =>1(0),       --
  5 =>1(1),       --A
  6 =>1(0),       --
  7 =>1(0),       --
  -- OLAC
  8 =>1(0),       --
  9 =>1(0),       --
  10 =>1(17),     --R
  11 =>1(0),      --
  -- LACO
  12 =>1(0),      --
  13 =>1(0),      --
  14 =>1(0),      --
  15 =>1(5));     --E
  
```

Implementarea acestei animații este asemănătoare cu deplasările. Într-o memorie nouă tot **ROM 16x7**, s-au pus cele **4 litere** ale cuvântului care vrem să apară și **3 spații**. Variabila **i** crește la fiecare secundă cu 4 unități afișând următoarele **4 adrese** care conțin litera următoare pe o poziție următoare și 3 spații.

În pozele de mai sus afișăm cuvântul **mere**

În pozele următoare, folosind cele 4 animații am afișat propoziția : **Opel este o mașină**



Alfabetul pe litere

